

Advancing Web Element Localization: Enhancements to the VON Similo-LLM

Nguyen Ba Hoang Anh¹, Nguyen Thi Cat Tuong², Pham Hong
Phuc³, Nguyen Quoc Dai⁴

¹22021100@vnu.edu.vn.

²22021207@vnu.edu.vn.

³22021220@vnu.edu.vn.

⁴20020389@vnu.edu.vn.

Abstract

Fragile web tests, primarily caused by locator breakages, are a persistent challenge in web development. Hence, researchers have proposed techniques for web-element re-identification in which algorithms utilize a range of element properties to relocate elements on updated versions of websites based on similarity scoring. In this essay, we proposed an extension of Similo and Von Similo and a strategy which reduces the computational overhead of the large language model. First, we extend the Similo feature set with two features to better capture the nature of elements webs. Second, we introduce adaptive weights which consider the contribution of attributes based on its behaves such as navigation, form input, button, text content, or media. Moreover, we propose an efficiency strategy to reduce the cost of LLMs. Our method achieves comparable accuracy compared to baselines while significantly decreasing the execution time of LLMs on the banchmark of 50 web applications. Code is available at <https://github.com/20020389/SimiloLLM>.

Keywords: Web Element Localization.

1 Introduction

Automated end-to-end (E2E) web tests created with tools such as Selenium are renowned for being fragile as the web application under test evolves [1, 2]. Researchers have singled out web element locators as the main cause of fragility [1, 3]. Locators

are commands used by test automation tools to identify elements on a web page, hanging on specific properties found in the Document Object Model (DOM), such as the element’s identifier, XPath, or text. Locator breakages are mainly caused by code changes in the web application. Given the short release cycles and advancements in modern websites, such breakages occur frequently. This poses a significant challenge for automated testing, as it requires manual fixing of tests before they can be executed again. This process is time-consuming and frustrating for testers. As a result, test suites are often abandoned [4], as the effort outweighs the benefits.

To mitigate these issues and minimize the number of fragile tests, researchers have proposed automated algorithms that produce robust locator [5, 6, 8]. The current state-of-the-art approach for locating a web element corresponding to a specific locator is the Similo algorithm [9]. This algorithm calculates a similarity score based on multiple properties of web elements to re-identify the target element among a set of candidates, which consist of all elements on an updated version of the website. The algorithm selects the candidate with the highest similarity score as the new target element. The original algorithm has been extended in two follow-up works, namely VON Similo [10] and LLM VON Similo [11]. The former extends the original Similo algorithm by comparing groups of visually overlapping elements on a website instead of singular elements. The latter leverages a large language model to improve the algorithm’s accuracy.

In this essay, we focus on the Similo algorithm [9] and its extensions because recent systematic literature reviews on web application testing [12] identify Similo as the current state-of-the-art approach for repairing broken web test cases. Similo has shown superior performance compared to baseline methods such as Robula+, Vista, and WATER.

Our essay makes the following contributions:

- We add two semantic features, **page region** and **functional group** to leverage the location and role information of each element.
- We define an adaptive weight set to adapt attribute importance, lead to the increase of accuracy compared to baselines.
- Efficient LLM strategy: We introduce Normalized Similarity Score (NSS) to decide whether the elements could be a candidate to ask LLMs, directly reducing overall LLMs calls with minimal accuracy loss.

2 Background

2.1 Similo

Similo requires a working version of the web application where the web element can be identified. Given the original web element T and its properties $T.a_1, \dots, T.a_n$, as well as all new web elements $C_1, \dots, C_n$, of which one should be the original one C' , Similo computes a similarity score between T and each $C_1, \dots, C_n$ individually:

$$\text{Similo}(T, C) = \sum_{i \in \# \text{properties}} \text{similarity}(C.a_i, T.a_i) \cdot c_i$$

Here, c_i is a property-specific weight based on the COLOR study, determining how effective, stable, and unique specific properties are. Stable properties, such as tag and name, are assigned a weight of 1.5, while non-stable properties receive a weight of 0.5. The similarity function calculates a score ranging from $[0, 1]$ on how similar both values are using a predetermined algorithm. This score is used to generate a ranking of all web elements based on their similarity with the original element, enabling developers to choose the element with the highest similarity score or retain all elements above a certain threshold.

The specific properties used in the Similo algorithm are selected based on the locator types supported by the Selenium WebDriver API (id, name, class, tag, link text, partial link text, XPath, and CSS) for native element location. The authors also consider the locators chosen by Selenium IDE, including id, link text, name, and various XPaths. Additional properties are chosen based on the results of the COLOR study and the WATER study. The authors selected all DOM-based properties from this list and excluded properties extracted from the visual user interface.

Properties with slight differences between versions, such as class, links, and XPaths, are compared using the Levenshtein distance. On the other hand, properties that tend to change completely, such as tag and id, are compared using equality. Integer-based properties are compared using the Euclidean distance.

2.2 VON Similo

The idea behind VON Similo is that web elements often consist of multiple parts. The DOM orders its elements so that a node's children are inside the area the parent occupies on the visually rendered website. They appear as one visual unit to the user and can be interacted with as one unit.

For example, a button might contain a button tag, an icon, and text. It does not matter which exact part the user clicks; the button will be triggered. When two elements have a considerable overlap, meaning they share a large part of their occupied area, they are likely to be part of the same visual unit. This unit is called visually overlapping nodes, or VON.

When a web element is moved to a different location on the website, the nodes in its visual overlap are often moved as well. The combination of elements moved together provides a more unique fingerprint than a single web element would. The situation is similar when a node is changed; its visual overlap might stay the same, making it easier to identify the modified element by the combination of elements in its visual overlap.

The improved algorithm in VON Similo leverages this heuristic to identify elements more reliably. When a target element needs to be found on a new website version, the algorithm first identifies the visual overlap of the target element on the baseline version. Then, it iterates over all candidates on the updated version and calculates their respective visual overlaps. In the last step, a score is calculated between the target visual overlap of nodes and all potential candidate visual overlaps.

The score is calculated between two elements, but includes all the property values of the visually overlapping nodes. For each property all possible combinations of values are compared and the maximum similarity is weighted and added to the total score.

Similar to Similo, the candidate with the highest score is returned as the target element in the new version.

To calculate the score, we need to define the visual overlap of a node. The VON Similo paper defines that two web elements E_1 and E_2 are visually overlapping if the following two conditions are met:

1. The areas R_1 and R_2 , which their respective rectangles occupy on the screen in pixels, intersect to a certain degree. In other words if $\frac{R_1 \cap R_2}{R_1 \cup R_2}$ is above a certain threshold, which should be chosen in a way that balances:
 - (a) accidentally grouping elements that do not belong together, when the threshold is chosen too loose, and there is no significant overlap, and
 - (b) not recognizing two visually overlapping nodes as such by choosing the threshold too high.

The authors propose a threshold of 0.85.

2. The center of E_2 is located inside of R_1 .

In addition the LLM VON Similo paper introduces another metric to determine if E_1 and E_2 are visually overlapping based of the nodes visual text and XPath, which is used in addition to the algorithm used in VON Similo. We will called this approach textual overlap, and it is defined as:

1. The visible text of both elements is not null and their content is case-sensitive equal.
2. The absolute XPath of E_1 is a prefix of the absolute XPath of E_2 , e.g., E_2 is a child of E_1 in the DOM.

After calculating all overlapping elements $E_1, \dots, E_n$ for one element E , we replace the properties $E.a_1, \dots, E.a_m$ of that element with lists of property values of the overlapping nodes.

2.3 LLM VON Similo

LLM VON Similo is the latest iteration of the Similo algorithm. The algorithm first uses VON Similo to rank all elements on the website by their score. It then takes the top ten visual overlaps, as the target element is most likely among them.

The algorithm then utilizes GPT-4 to find the target element among the pre-selected candidates. The algorithm converts the ten elements and the target element into JSON format. It then sends a request to the large language model with the target and the ten candidates, asking it to identify the target's match. The LLM responds with a single number, indicating the target's position in the list of candidate elements.

Since LLM VON Similo relies on proprietary OpenAI APIs and uses an unspecified version of GPT-4 with unknown temperature and configuration settings, its results cannot be reliably reproduced in a controlled experimental environment. Therefore, we do not attempt a full replication of this method in this essay. Rather, in our replication, we only consider the benchmark used in LLM VON Similo and the VON Similo implementation and test the other algorithms against it.

3 The Proposed Method

To mitigate the problems associated with threats, we propose the extension of the original Similo and VON Similo algorithms. Moreover, we introduce a hybrid strategy that leverages both.

3.1 Extending the feature set

While the original Similo algorithm uses 14 features deriving from DOM properties, we extend this set with two semantic features. The added features could bring more information about element’s function and location.

3.1.1 Page region

First, we use the *page_region* feature to capture the location of an element on each web and takes value from (header, footer, main, sidebar). To extract this feature, we employ an approach as follows: we check the XPath patterns (for example, if “header” or “nav” appears in the XPath, consider it as header), then we return to positional information corresponding to each element. This feature could solve an issue of original Similo: while pixel coordinates of elements are different under web changes, the information of semantic locations remain holds.

3.1.2 Functional group

We use another feature *functional_group* to categorize elements into five classes according to the context: form, navigation, button, text, media. We classify elements based on HTML tag semantics as follows:

Form: input, textarea, select, form.

Navigation: nav, ul, li.

Button: button, a.

Text: div, span, p, h1–h6, article, section.

Media: img, video, audio.

This feature serves two purposes. First, it provides a semantic constraint which could prevent the inconsistent between elements’ functions, for example, it could prevent **<button>** elements from matching with **<p>** elements despite these elements have similar text content. Second, it serves as an information for context detection when previous rules are not enough to determine the context of elements.

3.2 Modifying appropriate weights

We observe that Similo algorithm uses static weights derived from the REF study [9], which stabilize across diverse web applications. However, it is necessary to adapt weights according the element type. For example:

Form inputs: **id** and **name** elements are important, as they are often stable during web changes. However, the **visible_text** (label) usually changes because of localization.

Navigation links: **href** (URL) and **visible_text** (link text) are primary identifiers according the web, so that the position is not reliable.

Buttons: **visible_text** is the primary identifier because users recognize buttons by

Table 1: Context-Specific Weight Profiles (Selected Features)

Feature	Baseline	Form	Navigation	Button	Text	Media
<code>tag</code>	1.5	1.5	1.0	1.5	1.0	1.8
<code>id</code>	1.5	1.8	1.0	1.3	1.0	1.3
<code>name</code>	1.5	2.0	0.8	1.0	0.8	1.0
<code>href</code>	0.5	0.3	2.0	0.5	0.3	0.3
<code>visible_text</code>	1.5	1.2	1.8	1.8	2.5	0.8
<code>location</code>	0.5	0.5	0.3	0.5	0.5	0.5
<code>functional_group</code>	1.2	2.2	2.0	2.0	1.5	2.0
<code>page_region</code>	1.0	0.8	1.5	0.9	1.2	1.0

their labels.

The modified values of weights. For each context, we define a weight profile W_{ctx} to evaluate the importance of features. Table 1 presents our adaptive weights.

Specifically, the modified weight reflect feature stability and importance each context. For *form* elements, we increase `id` weights ($1.5 \rightarrow 1.8$) and `name` weights ($1.5 \rightarrow 2.0$) compare to original Similo since these attributes are often stable, while decreasing `visible_text` weights ($1.5 \rightarrow 1.2$) since it could easily change with localization. For *navigation* elements, we increase `href` tag ($0.5 \rightarrow 2.0$) since these URLs are primary identifiers. For *media* elements, `tag` and `alt` are essential to distinguish image/video types and provide semantic information.

3.3 Efficient strategy for LLMs

Although large language models can solve the issues which relate to semantic understanding, it also introduces significant computational overhead. In this work, we only select low-confidence cases and it could guarantee high accuracy while reducing executions of LLM.

Specifically, we propose a metric to quantify match confidence as follows. For a target element T and candidate C , we defined the Normalized Similarity Score (NSS) as follows:

$$\text{NSS}(T, C) = \frac{\text{Score}_{\text{ctx}}(T, C)}{\text{MaxScore}_{\text{ctx}}(T, C)} \quad (1)$$

where $\text{Score}_{\text{ctx}}(T, C)$ is the context-aware similarity score and $\text{MaxScore}_{\text{ctx}}(T, C) = \sum_i w_{\text{ctx}, i}$ is the maximum score when all available features match perfectly.

Clearly that NSS ranges from 0 to 1, representing the percentage of maximum similarity. Values close to 1.0 indicate high match confidence, while values below 0.7 suggest that these elements require LLM to output last decision.

Our filtering strategy can demonstrate as follows: given an element T and candidate set $\mathcal{C}$, we first detect the context of this element and apply the corresponding weight in Table 1 to compute similarity scores for all candidates. Second, we select the top candidate C^* with high score and calculate Normalized Similarity Score for these elements.

If $NSS \geq \tau$ (default: 0.85), we surpass C^* directly since this is a high confidence match. Otherwise, we extract the top 10 elements and send them to GPT-4.1 mini. In this essay, we preserve the same prompt structure as the baseline LLM VON Similo approach to require LLM re-ranking these candidates. The LLM would analyze element features in JSON format and returns the candidate id which seems most appropriate.

Threshold selection. We empirically evaluate $\tau \in \{0.7, 0.8, 0.85, 0.9, 0.95\}$ and find that $\tau = 0.85$ provides optimal accuracy.

4 Experiments

To demonstrate the effectiveness of the proposed method, the experiment answer the following research questions:

RQ1: *Do added features (page region and functional group) and our modified weights improve matching quality compared to baseline Similo?*

RQ2: Whether our proposed strategy reduce computational cost while maintaining high accuracy?

In addition to the usual accuracy metric, we also report results of methods using NSS equation above. The web applications chosen for this experiment are the same 50 websites used by Nass et al.[9] to evaluate previous versions of Similo, taken from the Alexa top 50 list.

4.1 Answer RQ1

We compare baseline Similo with our adjustment on feature set and weights (without LLMs). Overall, although the accuracy improves slightly, the NSS score increases dramatically from 66.38% to 78.84%, that means our work significantly improves in match quality. The NSS score increase also reveals that our method could produce more confident predictions: many matched elements which is correct have scores closer to the theoretical maximum, providing stronger evidence and greater robustness to other candidates.

Table 2: RQ1 Overall Performance Comparison

Method	Features	Accuracy	Mean NSS	Time (s)
Similo	14	91.29%	66.38%	29.7
Our (without LLMs)	16	91.54%	78.84%	31
Improvement	+2	+0.25%	+12.46%	

4.2 Answer RQ2

Table 3 compares methods with and without LLM re-ranking. Baseline Similo + LLM achieves 755/804 correct with 804 calls. Our method achieves 759/804 correct with 483 call with 40% fewer LLM invocations.

Table 3: RQ2 LLM Filtering Results

Configuration	Accuracy	Correct	LLM Calls	LLM Call Rate
Similo	91.29%	734/804	0	0%
Our	91.54%	736/804	0	0%
Similo + LLM	93.91%	755/804	804	100%
Our + LLMs ($\tau=0.85$)	94.40%	759/804	483	60.1%
Our + LLMs ($\tau=0.80$)	94.28%	758/804	351	43.7%

Table 4 analyzes different confidence thresholds. From this analysis, we observe that $\tau = 0.85$ is optimal choice. Lowering to $\tau = 0.80$ could introduces 3 errors while raising to $\tau = 0.90$ gains zero matches but requires 182 LLM calls more than that of $\tau = 0.85$ (665 vs 483).

Table 4: RQ2 Impact of Confidence Threshold

τ	Accuracy	Correct	Skipped	LLM Calls	High-Conf Acc
0.80	94.28%	758/804	453	351 (43.7%)	99.3% (450/453)
0.85	94.40%	759/804	321	483 (60.1%)	100% (321/321)
0.90	94.40%	759/804	139	665 (82.7%)	100% (139/139)

5 Related Works

We already discussed Similo [9], VON Similo [10], and LLM VON Similo [11], the first two were included in the empirical comparison conducted in this work. Besides the approaches investigated in this paper, several techniques have been proposed in literature to re-identify changed web elements in E2E web tests [1, 3].

Similarly to the algorithms investigated in this paper, COLOR [5] re-identifies changed web elements utilizing multiple properties of a target web element. Leotta et al. propose ROBULA+ [6] to create human-readable, short, and simple XPath expressions through an iterative refinement process of a generic XPath until the target element is uniquely identified. In follow-up work, the authors have extended the original algorithm by incorporating attribute robustness ranking, excluding fragile attributes, and adding textual information for improved stability [6].

In other works, the robust XPath locator problem is formulated as a graph search problem [13, 14]. Montoto et al. propose an algorithm for identifying elements in AJAX websites based on XPath expressions enriched with textual and attribute information.

LLM VON Similo is the latest iteration of the Similo algorithm [11]. The algorithm first uses VON Similo to rank all elements on the website by their score. It then takes the top ten visual overlaps, as the target element is most likely among them. The algorithm then utilizes GPT-4.1 mini to find the target element among the pre-selected candidates. The algorithm converts the ten elements and the target element

into JSON format. It then sends a request to the large language model with the target and the ten candidates, asking it to identify the target's match. The LLM responds with a single number, indicating the target's position in the list of candidate elements.

6 Conclusion

In this essay, we introduce an extension of Von Similo LLMs, a novel and efficiency framework to localize web elements. We first introduce two new features which better represent the information of each element. We further adjusted the weights depending on element context. While the accuracy improvement is small, we observe a significant increase in matching confidence (NNS score) on the benchmark with 50 web applications. Moreover, we propose a strategy to reduce expensive LLM usage with a threshold. Equipped with this strategy, our method can skip a substantial cases while keeping high accuracy, leading to fewer LLM calls under the same benchmark settings. Future work could learn weights automatically, and evaluate on more applications or datasets.

References

- [1] Hammoudi, M., Rothermel, G., Tonella, P.: Why do record/replay tests of web applications break? In: *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST)*, pp. 180–190 (2016)
- [2] Leotta, M., García, B., Ricca, F., Whitehead, J.: Challenges of end-to-end testing with Selenium WebDriver and how to face them: A survey. In: *Proceedings of ICST*, pp. 339–350 (2023)
- [3] Stocco, A., Yandrapally, R., Mesbah, A.: Visual web test repair. In: *Proceedings of the ESEC/FSE*, pp. 1–12 (2018)
- [4] Christophe, L., Stevens, R., De Roover, C., De Meuter, W.: Prevalence and maintenance of automated functional tests for web applications. In: *IEEE International Conference on Software Maintenance and Evolution*, pp. 141–150 (2014)
- [5] Kirinuki, H., Tanno, H., Natsukawa, K.: COLOR: Correct locator recommender for broken test scripts using various clues in web applications. In: *Proceedings of SANER*, pp. 310–320 (2019)
- [6] Leotta, M., Stocco, A., Ricca, F., Tonella, P.: ROBULA+: An algorithm for generating robust XPath locators for web testing. *Journal of Software: Evolution and Process* 28(3), 177–204 (2016)
- [7] Leotta, M., Stocco, A., Ricca, F., Tonella, P.: Using multi-locators to increase the robustness of web test cases. In: *Proceedings of ICST*, pp. 1–10 (2015)

- [8] Choudhary, S. R., Zhao, D., Versee, H., Orso, A.: WATER: Web application test repair. In: *Proceedings of ETSE*, pp. 24–29 (2011)
- [9] Nass, M., Alégroth, E., Feldt, R., Leotta, M., Ricca, F.: Similarity-based web element localization for robust test automation. *ACM Transactions on Software Engineering and Methodology* 32(3) (2023)
- [10] Nass, M., Alégroth, E., Feldt, R., Coppola, R.: Robust web element identification for evolving applications by considering visual overlaps. In: *Proceedings of ICST*, pp. 258–268 (2023)
- [11] Nass, M., Alégroth, E., Feldt, R.: Improving web element localization by using a large language model. *Software Testing, Verification and Reliability* (2024)
- [12] Balsam, S., Mishra, D.: Web application testing-challenges and opportunities. *Journal of Systems and Software* 219, 112186 (2025)
- [13] Leotta, M., Ricca, F., Tonella, P.: SIDEREAL: Statistical adaptive generation of robust locators for web testing. *Software Testing, Verification and Reliability* 31 (2021)
- [14] Leotta, M., Stocco, A., Ricca, F., Tonella, P.: Meta-heuristic generation of robust XPath locators for web testing. In: *Proceedings of SBST*, pp. 36–39 (2015)